# Chapter 1

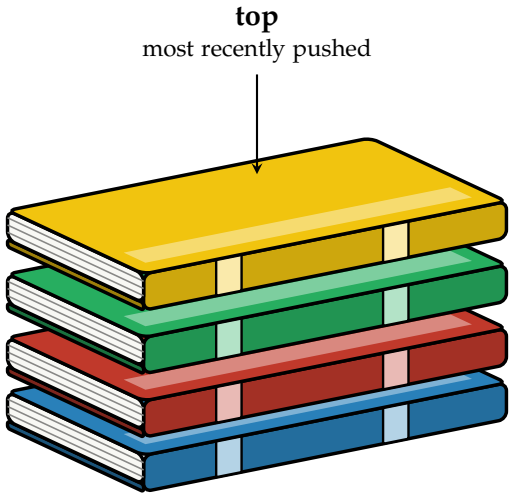
# Stacks

A *stack* is a linear data structure that only allows access at one end, called the *top*.

Data comes off in the reverse order it went on: the last thing pushed onto the stack is the first thing popped off. This ordering is called *LIFO*, for "last in, first out."

You have already seen a stack in a previous workbook, even if it wasn't called one: the call stack from the Introduction to C chapter behaves exactly this way, with each function call pushing a new frame on top and returning popping it back off.

We can think of this like a stack of books: you can only access the book on top, and you can only see the cover of the top book. If you want to get to a book underneath, you have to remove the books above it first.




Every stack, no matter how it is built underneath, supports the same handful of operations:

- `push(value)` put a new value on top of the stack.
- `pop()` remove and return the value on top of the stack.
- `peek()` look at the value on top of the stack, without removing it.
- `isEmpty()` true if the stack has nothing in it.

Notice that we can only use the value on top of the stack, no matter how large it is.

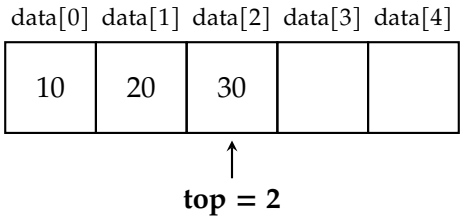
We are going to build a stack two different ways: once backed by a fixed-size array, and once backed by the linked-list nodes from the previous chapter. Both give you `push/pop/peek/isEmpty`, but they make very different tradeoffs underneath.

## 1.1 Array-Based Stack

The simplest way to build a stack is on top of a plain array. We keep a fixed-size buffer and a single integer, `top`, that tracks the index of the top element. An empty stack is represented by `top == -1`.

```
const int CAPACITY = 5;

struct ArrayStack {
    int data[CAPACITY];
    int top;   // index of the top element; -1 means empty
};
```




`push` and `pop` only ever touch the `top` index and whatever cell it points at, so both run in constant time, $O(1)$. The catch is `CAPACITY`: once the array is full, there is nowhere left to push, so a correct implementation has to check for and reject that case rather than writing past the end of the array.

```
bool isEmpty(const ArrayStack& s) {
    return s.top == -1;
}

bool isFull(const ArrayStack& s) {
    return s.top == CAPACITY - 1;
}

void push(ArrayStack& s, int value) {
    // prevent pushing onto a full stack
    if (isFull(s)) {
        std::cout << "push(" << value << ") failed: stack overflow" << std::endl;
        return;
    }
    s.top++;
```

```
    s.data[s.top] = value;
}

int pop(ArrayStack& s) {
    // prevent popping from an empty stack
    if (isEmpty(s)) {
        std::cout << "pop() failed: stack underflow" << std::endl;
        return -1;
    }
    int value = s.data[s.top];
    s.top--;
    return value;
}
```

Notice that `pop` does not actually erase `s.data[s.top]`, it just moves `top` down by one. The old value is still sitting in the array, but nothing can reach it anymore, since every operation only ever looks at indices `0` through `top`.

The full, runnable version of this file, including a `main()` that demonstrates both overflow and underflow, is in your digital resources. In this version, we added memory address output to demonstrate that shows how the array is a contiguous block.

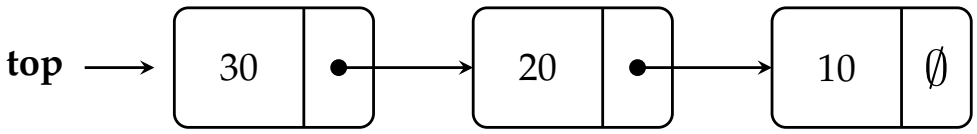
## 1.2 Linked-List Stack

The array-based stack has one real weakness: `CAPACITY` is fixed at compile time. If you don't know in advance how much data you'll be pushing, you either waste memory reserving more than you need, or you risk overflow. A stack built on the `Node` type from the Linked Lists chapter has no limit, since it grows and shrinks on the heap as needed. The downside is that each element takes more memory, since it has to store a pointer to the next node.

```
struct Node {
    int data;
    Node* next;
};

struct LinkedStack {
    Node* top;   // nullptr means empty
};
```

Instead of an array index, `top` is now a pointer to the first node. Pushing means creating a new node whose `next` points at the old top, then moving `top` to the new node; popping is the reverse.




```cpp
bool isEmpty(const LinkedStack& s) {
    return s.top == nullptr;
}

void push(LinkedStack& s, int value) {
    Node* n = new Node();
    n->data = value;
    n->next = s.top;    // the new node points at the old top
    s.top = n;          // the new node becomes the top
}

int pop(LinkedStack& s) {
    if (isEmpty(s)) {
        std::cout << "pop() failed: stack underflow" << std::endl;
        return -1;
    }
    Node* oldTop = s.top;
    int value = oldTop->data;
    s.top = oldTop->next;
    delete oldTop;
    return value;
}
```

Both `push` and `pop` still run in constant time (we will cover this in a future chapter), same as the array version, since neither one has to walk the list. But unlike the array version, `pop` here calls `delete` on the old top node; if it didn't, the node would sit on the heap forever.

The full version of this file is in your digital resource!

## 1.3 Array vs. Linked: Which One?

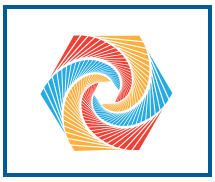
| | **Array-Based** | **Linked-Based** |
|---|---|---|
| Max size | Fixed at compile time | Limited only by available memory |
| Memory per element | Just the value | Value + one pointer |

If you know a safe upper bound on how much data you'll ever hold, the array version is the most efficient* choice. If you don't, the linked version is safer, since it will never overflow.

*This is a draft chapter from the Kontinua Project. Please see our website (https://kontinua.org/) for more details.*

Appendix A

# Answers to Exercises



# Index

LIFO, 1

stack, 1

top, 1